8
Inference Optimization

Deploying LLMs is challenging due to their significant computational and memory requirements. Efficiently running these models necessitates the use of specialized accelerators, such as GPUs or TPUs, which can parallelize operations and achieve higher throughput. While some tasks, like document generation, can be processed in batches overnight, others require low latency and fast generation, such as code completion. As a result, optimizing the inference process – how these models make predictions based on input data – is critical for many practical applications. This includes reducing the time it takes to generate the first token (latency), increasing the number of tokens generated per second (throughput), and minimizing the memory footprint of LLMs.

Indeed, naive deployment approaches lead to poor hardware utilization and underwhelming throughput and latency. Fortunately, a variety of optimization techniques have emerged to dramatically speed up inference. This chapter will explore key methods like speculative decoding, model parallelism, and weight quantization, demonstrating how thoughtful implementations can achieve speedups of 2–4X or more. We will also introduce three popular inference engines (Text Generation Inference, vLLM, and TensorRT-LLM) and compare their features in terms of inference optimization.

In this chapter, we will cover the following topics:

- Model optimization strategies
- Model parallelism
- Model quantization

By the end of this chapter, you will understand the core challenges in LLM inference and be familiar with state-of-the-art optimization techniques, including model parallelism and weight quantization.

All the code examples from this chapter can be found on GitHub at https://github.com/PacktPublishing/LLM-Engineering.

Model optimization strategies

Most of the LLMs used nowadays, like GPT or Llama, are powered by a decoder-only Transformer architecture. The *decoder-only* architecture is designed for text-generation tasks. It predicts the next word in a sequence based on preceding words, making it effective for generating contextually appropriate text continuations.

In contrast, an *encoder-only* architecture, like BERT, focuses on understanding and representing the input text with detailed embeddings. It excels in tasks that require comprehensive context understanding, such as text classification and named entity recognition. Finally, the encoder-decoder architecture, like T5, combines both functionalities. The encoder

processes the input text to generate a context-rich representation, which the decoder then uses to produce the output text. This dual structure is particularly powerful for sequence-to-sequence tasks like translation and summarization, where understanding the input context and generating a relevant output are equally important.

In this book, we only focus on the decoder-only architecture, which dominates the LLM field.

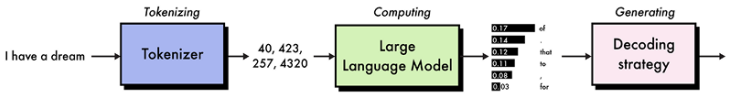

Figure 8.1 – Inference process with decoder-only models. We provide "I have a dream" as input and obtain "of" as output.

As shown in *Figure 8.1*, the basic inference process for a decoder-only model involves:

1. **Tokenizing** the input prompt and passing it through an embedding layer and positional encoding.
2. **Computing** key and value pairs for each input token using the multi-head attention mechanism.
3. **Generating** output tokens sequentially, one at a time, using the computed keys and values.

While *Steps 1* and *2* are computationally expensive, they consist of highly parallelizable matrix multiplication that can achieve high hardware utilization on accelerators like GPUs and TPUs.

The real challenge is that the token generation in *Step 3* is inherently sequential – to generate the next token, you need to have generated all previous tokens. This leads to an iterative process where the output sequence is grown one token at a time, failing to leverage the parallel computing capabilities of the hardware. Addressing this bottleneck is one of the core focuses of inference optimization.

In this section, we will detail several optimization strategies that are commonly used to speed up inference and reduce **Video Random-Access Memory (VRAM)** usage, such as implementing a (static) KV cache, continuous batching, speculative decoding, and optimized attention mechanisms.

KV cache

We saw that LLMs generate text token by token, which is slow because each new prediction depends on the entire previous context. For example, to predict the 100th token in a sequence, the model needs the context of tokens 1 through 99. When predicting the 101st token, it again needs the information from tokens 1 through 99, plus token 100. This repeated computation is particularly inefficient.

The **key-value (KV)** cache addresses this issue by storing key-value pairs produced by self-attention layers. Instead of recalculating these pairs for each new token, the model retrieves them from the cache, significantly speeding up the generation.

You can see an illustration of this technique in *Figure 8.2*:

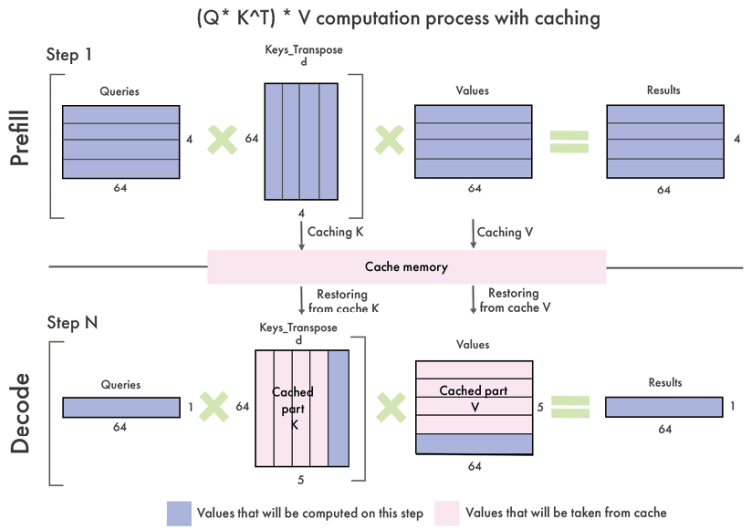

Figure 8.2 – Illustration of the KV cache

When a new token is generated, only the key and value for that single token need to be computed and added to the cache. The KV cache is an immediate optimization that is implemented in every popular tool and library. Some implementations maintain a separate KV cache for each layer of the model.

The size of the KV cache scales with the number of tokens (n_{tokens}) and several model dimensions, like the number of layers (n_{layers}), the number of attention heads (n_{heads}), their dimension ($\dim_{head}$), and the precision of the parameters in bytes (n_{bytes}):

$$size(KV\ cache) = 2 n_{tokens}\ n_{layers}\ n_{heads}\ \dim_{head}\ n_{bytes}$$

For a typical 7B parameter model using 16-bit precision, this exceeds 2 GB for high sequence lengths (higher than 2,048 tokens). Larger models with more layers and higher embedding dimensions will see even greater memory requirements.

Since the KV cache grows with each generation step and is dynamic, it prevents you from taking advantage of torch.compile, a powerful optimization tool that fuses PyTorch code into fast and optimized kernels. The *static KV cache* solves this issue by pre-allocating the KV cache size to a maximum value, which allows you to combine it with `torch.compile` for up to a 4x speedup in the forward pass.

To configure a model to use a static KV cache with the transformers library, follow these steps:

1. We import the tokenizer and the model we want to optimize:

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
model_id = "google/gemma-2b-it"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto")
```

Quick tip: Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (**1**) to quickly copy code into your coding environment, or click the **Explain** button (**2**) to get the AI assistant to explain a block of code to you.

The next-gen Packt Reader is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting https://www.packtpub.com/unlock/9781836200079.

2. To implement the static cache, we change the cache implementation in the model's generation config to `static`:

```
model.generation_config.cache_implementation = "static"
```

3. Now that our KV cache is static, we can compile the model using torch.compile:

```
compiled_model = torch.compile(model, mode="reduce-overhead", fullgraph=True)
```

4. We tokenize an input question, "`What is 2+2?`", and store it on a GPU if available (if not, we store it on the CPU):

```
device = "cuda" if torch.cuda.is_available() else "cpu"
inputs = tokenizer("What is 2+2?", return_tensors="pt").to(device)
```

5. Let's use the `generate()` method to get the model's output and decode it with `batch_decode()` to print its answer:

```
outputs = model.generate(**inputs, do_sample=True, temperature=0.7, max_new_toke
print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
['What is 2+2?\n\nThe answer is 4. 2+2 = 4.']
```

This returns a list containing both the input and output, correctly answering our question.

Note that the static cache doesn't work with all architectures. For details on which architectures are supported, check out the transformers documentation.

Efficiently managing the KV cache is essential, as it can quickly exhaust available GPU memory and limit the batch sizes that can be processed. This has motivated the development of memory-efficient attention mechanisms and other techniques, which we will cover in the last section.

Continuous batching

Batching, or processing multiple inference requests simultaneously, is a standard approach to achieve high throughput. Larger batch sizes spread

out the memory cost of model weights and transfer more data to the GPU at once, better saturating its parallel compute capacity.

However, decoder-only models pose a particular challenge due to the high variability in input prompt lengths and desired output lengths. Some requests may have short prompts and only need a one-word answer, while others may input a lengthy context and expect a multi-paragraph response.

With traditional batching, we would have to wait for the longest request in a batch to complete before starting a new batch. This leads to under-utilization as the accelerator sits partly idle waiting for a straggling request to finish. *Continuous batching*, also known as `in-flight` batching, aims to prevent idle time by immediately feeding a new request into the batch as soon as one completes.

The batching process begins the same – by filling the batch with initial requests. But as soon as a request completes its generation, it is evicted from the batch and a new request takes its place. This way, the accelerator is always processing a full batch, leading to maximally efficient hardware utilization. An additional consideration is the need to periodically pause the generation process to run prefill, or the embedding and encoding of waiting requests. Finding the optimal balance between generation and prefill requires some tuning of the waiting-served ratio hyperparameter.

Continuous batching is natively implemented in most inference frameworks, like Hugging Face's **Text Generation Inference** (**TGI**), vLLM, and NVIDIA TensorRT-LLM.

Speculative decoding

Another powerful optimization technique is *speculative decoding*, also called assisted generation. The key insight is that even with continuous batching, the token-by-token generation process fails to fully saturate the parallel processing capabilities of the accelerator. Speculative decoding aims to use this spare compute capacity to predict multiple tokens simultaneously, using a smaller proxy model (see *Figure 8.3*).

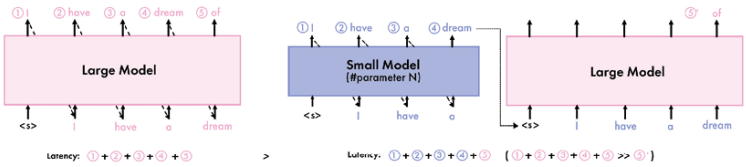

Figure 8.3 – Illustration of traditional decoding (left) and speculative decoding (right)

The general approach is:

- Apply a smaller model, like a distilled or pruned version of the main model, to predict multiple token completions in parallel. This could be 5–10 tokens predicted in a single step.
- Feed these speculative completions into the full model to validate which predictions match what the large model would have generated.

- Retain the longest matching prefix from the speculative completions and discard any incorrect tokens.

The result is that, if the small model approximates the large model well, multiple tokens can be generated in a single step. This avoids running the expensive large model for several iterations. The degree of speedup depends on the quality of the small model's predictions – a 90% match could result in a 3–4X speedup.

It is crucial that both models use the same tokenizer. If this is not the case, the tokens generated by the draft model will not align with those produced by the large model, making them incompatible. Let's implement this using the transformers library. In this example, we will use two Qwen1.5 models from Alibaba Cloud: a 1.8B version as the main model, and a 0.5B version as the draft model. Note that, if you have enough VRAM, you can use much larger models like 14B, 32B, 72B, or 110B as the main model. Here, we're limited by the VRAM of the T4 GPU in Google Colab, but to get the maximum speedup, the assistant model should be much smaller than the large model.

Here's a step-by-step guide to implement speculative decoding:

1. We load the tokenizer and both models:

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
model_id = "Qwen/Qwen1.5-1.8B-Chat"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto")
draft_model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen1.5-0.5B-Chat", dev
```

2. We then tokenize the same input and store it in the accelerator, if available:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
inputs = tokenizer("What is 2+2?", return_tensors="pt").to(device)
```

3. We can now use `model.generate()` with the argument `assistant_model` to enable speculative decoding:

```
outputs = model.generate(**inputs, do_sample=True, assistant_model=draft_model,
print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
['What is 2+2? 2 + 2 equals 4!']
```

The speedup in this small example is not significant, but it is clearly noticeable with bigger models.

Prompt lookup decoding is a variant of speculative decoding, tailored to input-grounded tasks like summarization where there is often overlap between the prompt and output. Shared n-grams are used as the LLM candidate tokens. We can enable prompt lookup decoding by using the `prompt_lookup_num_tokens` parameter in `model.generate()`:

```
outputs = model.generate(**inputs, prompt_lookup_num_tokens=4)
```

By combining the static KV cache with torch.compile, implementing continuous batching, and leveraging speculative decoding techniques, LLMs can see inference speedups of 2–4x or more with no loss in quality.

Another approach to creating a small proxy model consists of jointly fine-tuning a small model alongside a large model for maximum fidelity. A representative technique here is Medusa, which inserts dedicated speculation heads into the main model. The Medusa-1 approach fine-tunes these speculation heads while freezing the large model, while the Medusa-2 approach jointly fine-tunes both the speculation heads and the large model. The Medusa method has demonstrated impressive results, enabling a 70M parameter model to closely approximate the performance of a 7B parameter model on a range of tasks. Speculative decoding is natively supported by TGI.

Optimized attention mechanisms

The Transformer architecture is based on the attention mechanism, which scales quadratically with the number of input tokens (or sequence length). This is particularly inefficient for longer sequences, where the size of the KV cache can blow up.

Introduced by Kwon, Li, et al. (2023), *PagedAttention* addresses these memory challenges by drawing inspiration from virtual memory and paging in operating systems. It partitions the KV cache into blocks, eliminating the need for contiguous memory allocation. Each block contains the keys and values for a fixed number of tokens. During attention computation, the PagedAttention kernel efficiently fetches these blocks, regardless of their physical memory location.

This partitioning allows for near-optimal memory utilization. This is useful for batching more sequences together, which increases throughput and GPU utilization. Moreover, `PagedAttention`'s block-based approach naturally supports memory sharing across multiple output sequences generated from the same prompt. This is particularly advantageous in parallel sampling and beam search, where the same prompt is used to generate multiple outputs. The shared memory blocks reduce redundant computations and memory usage, cutting the memory overhead by up to 55% and improving throughput by up to 2.2x, according to the authors. The vLLM library received the first implementation of PagedAttention. Since then, PagedAttention has also been implemented in TGI and TensorRT-LLM.

Another popular option is *FlashAttention-2*. Developed by Tri Dao (2023), it introduced several key innovations that are designed to address the quadratic runtime and memory constraints in traditional attention. By dividing input and output matrices into smaller blocks, FlashAttention-2 ensures that these blocks can fit into the GPU's on-chip SRAM, which is much faster than high-bandwidth memory. This approach significantly reduces the frequency of data transfers between the GPU's main memory and its processing units. This is combined with online softmax, which computes the softmax function independently for each block of the attention scores matrix, rather than for the entire matrix at once. By maintaining a running maximum and a running sum of exponentials, FlashAttention-2 can calculate attention probabilities without needing to store large intermediate matrices.

Additionally, FlashAttention-2's online softmax computation enables block-wise processing, maintaining accuracy while significantly reducing memory requirements. This is particularly important for training, where the recomputation of intermediate values (instead of storing them) in the backward pass reduces memory usage from quadratic to linear, in relation to sequence length.

Unlike PagedAttention, FlashAttention-2 can easily be used with the transformers library through the `attn_implementation` parameter:

1. Install the `flash-attn` library with `--no-build-isolation` so that we don't install the dependencies:

```
pip install flash-attn --no-build-isolation
```

2. To use FlashAttention-2 for inference, specify `flash_attention_2` in the `attn_implementation` parameter when loading a model. For example, this is how to load Mistral-7B-Instruct-v0.3 with FlashAttention-2:

```
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-Instruct-v0.3",
    attn_implementation="flash_attention_2",
)
```

The techniques presented in this section focused on improving the model's efficiency in processing tokens. In the next section, we will discuss how to distribute our model and calculations across multiple GPUs.

Model parallelism

Model parallelism allows you to distribute the memory and compute requirements of LLMs across multiple GPUs. This enables the training and inference of models too large to fit on a single device, while also improving performance in terms of throughput (tokens per second).

There are three main approaches to model parallelism, each involving splitting the model weights and computation in different ways: *data parallelism*, *pipeline parallelism*, and *tensor parallelism*. Although these approaches were originally developed for training, we can reuse them for inference by focusing on the forward pass only.

Data parallelism

Data parallelism (DP) is the simplest type of model parallelism. It involves making copies of the model and distributing these replicas across different GPUs (see *Figure 8.4*). Each GPU processes a subset of the data simultaneously. During training, the gradients calculated on each GPU are averaged and used to update the model parameters, ensuring that each replica remains synchronized. This approach is particularly beneficial when the batch size is too large to fit into a single machine or when aiming to speed up the training process.

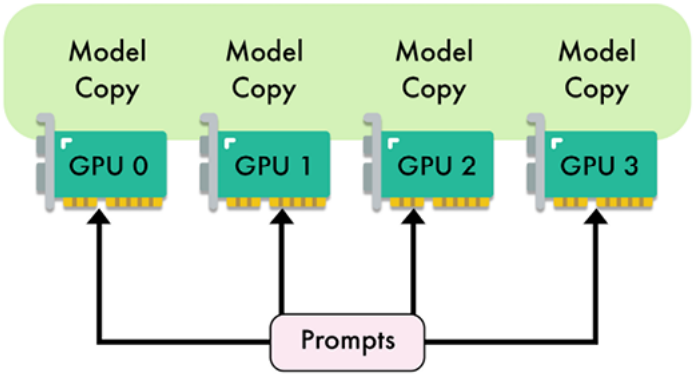

Figure 8.4 – Illustration of data parallelism with four GPUs

During inference, DP can be useful for processing concurrent requests. By distributing the workload across multiple GPUs, this approach helps reduce latency, as multiple requests can be handled simultaneously. This concurrent processing also increases throughput, since a higher number of requests can be processed at the same time.

However, the effectiveness of DP is limited by the model size and the communication overhead between GPUs. Indeed, replicating the model's parameters on each GPU is inefficient. This means that this technique only works when the model is small enough to fit into a single GPU, leaving less room for input data and thus limiting the batch size. For larger models or when memory is a constraint, this can be a significant drawback.

Typically, DP is mainly used for training, while pipeline and tensor parallelism are preferred for inference.

Pipeline parallelism

Introduced by Huang et al. in the GPipe paper (2019), **pipeline parallelism (PP)** is a strategy for distributing the computational load of training and running large neural networks across multiple GPUs.

Unlike traditional DP, which replicates the entire model on each GPU, pipeline parallelism partitions the model's layers across different GPUs. This approach allows each GPU to handle a specific portion of the model, thereby reducing the memory burden on individual GPUs.

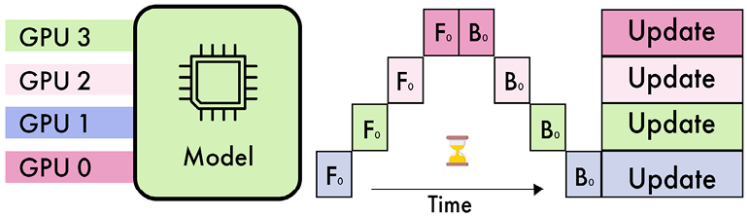

Figure 8.5 – Illustration of pipeline parallelism with four GPUs

As shown in *Figure 8.5*, in a typical four-way pipeline parallel split, the model is divided into four segments, with each segment assigned to a different GPU. The first 25% of the model's layers might be processed by GPU 1, the next 25% by GPU 2, and so on. During the forward pass, activations are computed and then passed along to the next GPU. For training, the

backward pass follows a similar sequence in reverse, with gradients being propagated back through the GPUs. The number of GPUs is often referred to as the degree of parallelism.

The primary advantage of pipeline parallelism is its ability to significantly reduce the memory requirements per GPU. However, this approach introduces new challenges, particularly related to the sequential nature of the pipeline. One of the main issues is the occurrence of "pipeline bubbles." These bubbles arise when some GPUs are idle, waiting for activations from preceding layers. This idle time can reduce the overall efficiency of the process.

Micro-batching was developed to mitigate the impact of pipeline bubbles. By splitting the input batch into smaller sub-batches, micro-batching ensures that GPUs remain busier, as the next sub-batch can begin processing before the previous one is fully completed.

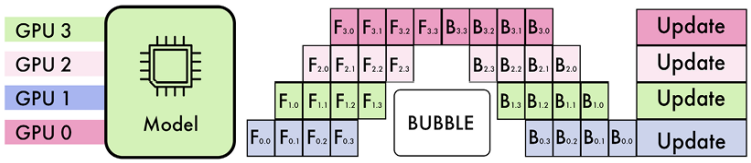

Figure 8.6 – Illustration of pipeline parallelism with micro-batching.

Figure 8.6 shows an example of pipeline parallelism with micro-batching. In this example, the pipeline has four stages (**F0**, **F1**, **F2**, **F3**), and the input batch is divided into four micro-batches. GPU 0 will process forward paths **F0**,0, **F0**,1, **F0**,2, and **F0**,3, sequentially. Once **F0**,0 is complete, GPU 1 can immediately start processing **F1**,0 and so on. After completing these forward passes, GPU 0 waits for the other GPUs to finish their respective forward computations before starting the backward paths (**B0**,3, **B0**,2, **B0**,1, and **B0**,0).

Pipeline parallelism is implemented in distributed training frameworks like Megatron-LM, DeepSpeed (ZeRO), and PyTorch through the dedicated **Pipeline Parallelism for PyTorch** (**PiPPy**) library. At the time of writing, only certain inference frameworks like TensorRT-LLM support pipeline parallelism.

Tensor parallelism

Introduced by Shoeby, Patwary, Puri et al. in the Megatron-LM paper (2019), **tensor parallelism** (**TP**) is another popular technique to distribute the computation of LLM layers across multiple devices. In contrast to pipeline parallelism, TP splits the weight matrices found in individual layers. This enables simultaneous computations, significantly reducing memory bottlenecks and increasing processing speed.

In TP, large matrices, such as the weight matrices in MLPs or the attention heads in self-attention layers, are partitioned across several GPUs. Each GPU holds a portion of these matrices and performs computations on its respective slice.

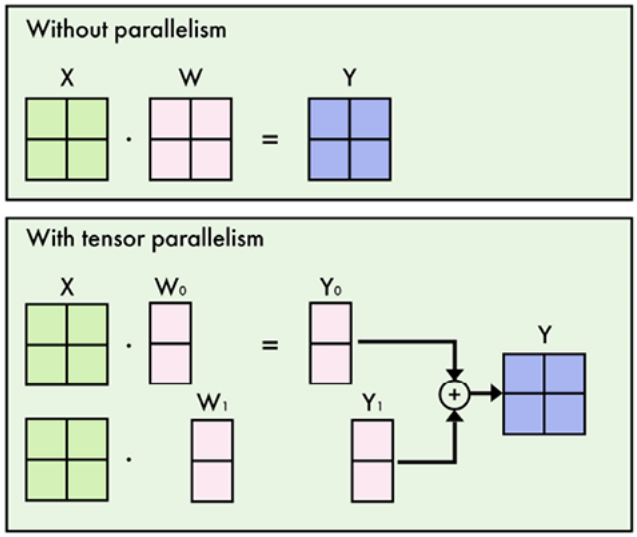

Figure 8.7 – Illustration of column-wise tensor parallelism in an MLP layer (W)

For instance, in an MLP layer, the weight matrix is divided so that each GPU processes only a subset of the weights (see *Figure 8.7*). The inputs are broadcast to all GPUs, which then independently compute their respective outputs. The partial results are then aggregated through an all-reduce operation, combining them to form the final output.

In the context of self-attention layers, TP is particularly efficient due to the inherent parallelism of attention heads. Each GPU can compute a subset of these heads independently, allowing the model to process large sequences more effectively. This makes TP more efficient than pipeline parallelism, which requires waiting for the completion of previous layers.

Despite its advantages, TP is not universally applicable to all layers of a neural network. Layers like LayerNorm and Dropout, which have dependencies spanning the entire input, cannot be efficiently partitioned and are typically replicated across devices instead. However, these operations can be split on the sequence dimension of the input instead (sequence parallelism). Different GPUs can compute these layers on different slices of the input sequence, avoiding replication of weights. This technique is limited to a few specific layers, but it can provide additional memory savings, especially for very large input sequence lengths.

Moreover, TP necessitates high-speed interconnects between devices to minimize communication overhead, making it impractical to implement across nodes with insufficient interconnect bandwidth.

TP is also implemented in distributed training frameworks like Megatron-LM, DeepSpeed (ZeRO), and PyTorch (FSDP). It is available in most inference frameworks, like TGI, vLLM, and TensorRT-LLM.

Combining approaches

Data, tensor, and pipeline parallelisms are orthogonal techniques that can be combined. *Figure 8.8* illustrates how a given model can be split according to each approach:

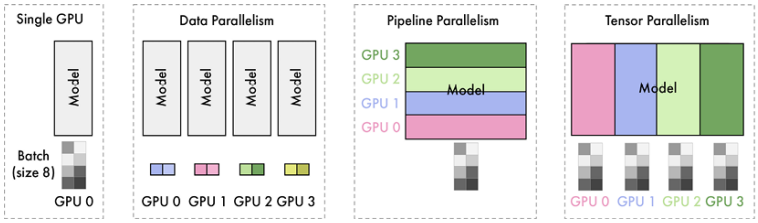

Figure 8.8 – Illustration of the different model parallelism techniques

Combining these techniques can mitigate their respective issues. Pipeline parallelism provides the greatest memory reduction but sacrifices efficiency, due to pipeline bubbles. This may be ideal if the primary constraint fits the model in the GPU memory. In contrast, if low latency is paramount, then prioritizing tensor parallelism and accepting a larger memory footprint may be the better trade-off. In practice, a model may be split depth-wise into a few pipeline stages, with tensor parallelism used within each stage.

Balancing these tradeoffs and mapping a given model architecture onto available hardware accelerators is a key challenge in deploying LLMs.

Model quantization

Quantization refers to the process of representing the weights and activations of a neural network using lower-precision data types. In the context of LLMs, quantization primarily focuses on reducing the precision of the model's weights and activations.

By default, weights are typically stored in a 16-bit or 32-bit floating-point format (FP16 or FP32), which provides high precision but comes at the cost of increased memory usage and computational complexity. Quantization is a solution to reduce the memory footprint and accelerate the inference of LLMs.

In addition to these benefits, larger models with over 30 billion parameters can outperform smaller models (7B–13B LLMs) in terms of quality when quantized to 2- or 3-bit precision. This means they can achieve superior performance while maintaining a comparable memory footprint.

In this section, we will introduce the concepts of quantization, GGUF with `llama.cpp`, GPTQ, and EXL2, along with an overview of additional techniques. In addition to the code provided in this section, you can refer to AutoQuant (bit.ly/autoquant) to quantize their models using a Google Colab notebook.

Introduction to quantization

There are two main approaches to weight quantization: **Post-Training Quantization** (**PTQ**) and **Quantization-Aware Training** (**QAT**). PTQ is a straightforward technique where the weights of a pre-trained model are directly converted to a lower precision format without any retraining. While PTQ is easy to implement, it may result in some performance degradation. Conversely, QAT performs quantization during the training or fine-tuning stage, allowing the model to adapt to the lower precision weights. QAT often yields better performance compared to PTQ but requires additional computational resources and representative training data.

The choice of data type plays a crucial role in quantization. Floating-point numbers, such as **FP32**, **FP16** (half-precision), and **BF16** (brain floating-point), are commonly used in deep learning. These formats allocate a fixed number of bits to represent the `sign`, `exponent`, and `significand` (mantissa) of a number.

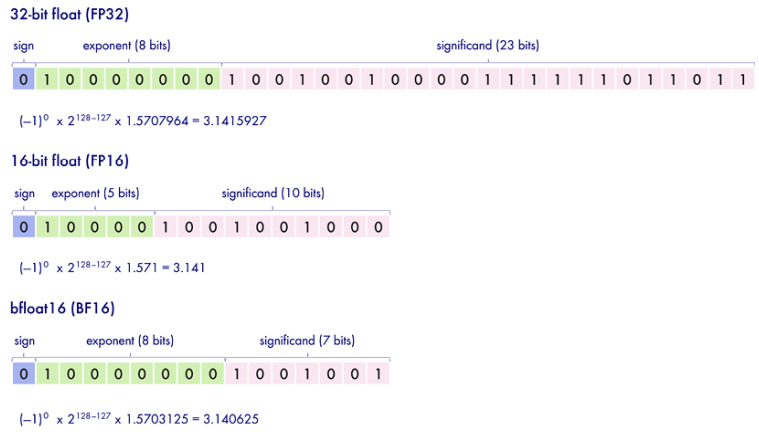

Figure 8.9 – Comparison the between FP32, FP16, and BF16 formats

A sign of 0 represents a positive number, while 1 indicates a negative number. Conversely, the exponent controls the range that is represented (big or small). Finally, the significand controls the precision of the number (the number of digits). The formula used to convert these representations into real numbers is:

$$(-1)^{\text{sign}} \times \text{base}^{\text{exponent}} \times \text{significand}$$

The data types shown in *Figure 7.7* display different tradeoffs, as illustrated with different representations of π (≈ 3.1415926535). FP32 uses 32 bits, providing high precision but also requiring more memory. Conversely, FP16 and BF16 use 16 bits, lowering the memory footprint at the cost of a lower precision. In general, neural networks prefer a bigger range than better precision, which is why BF16 is the most popular data type when the hardware supports it. For example, NVIDIA's Ampere architecture (A100, A30, etc.) supports BF16, but previous generations like Turing (T4, T40, etc.) do not.

However, we are not restricted to these three data types. Lower-precision data types, such as INT8 (8-bit integers), can be employed for quantization, further reducing the memory footprint. Naïve quantization techniques, such as *absolute maximum (absmax) quantization* and *zero-point quantization*, can be applied to convert `FP32`, `FP16`, or `BF16` weights to `INT8`, as illustrated in *Figure 8.10*:

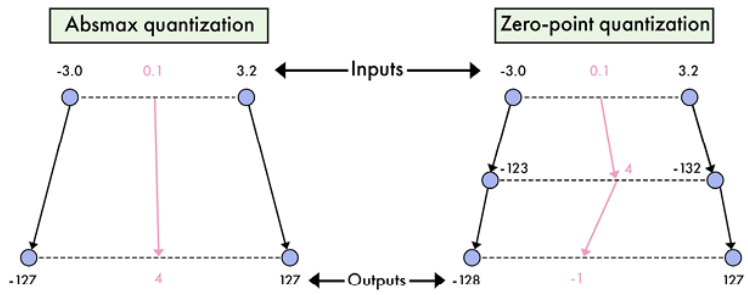

Figure 8.10 – Quantization of 0.1 in a [-3.0, 3.2] range with absmax quantization and zero-point quantization

Absmax quantization maps the original weights $\mathbf{X}$ to the range [-127, 127] by dividing them by the absolute maximum value of $\mathbf{X}$ and scaling them:

$$\mathbf{X}_{\text{quant}} = \text{round}\left(\frac{127 \cdot \mathbf{X}}{\max|\mathbf{X}|}\right)$$

For example, if our absolute maximum value is 3.2 (see *Figure 8.8*), a weight of 0.1 would be quantized to $\text{round}\left(\frac{127 \cdot 0.1}{3.2}\right) = 4$. To dequantize it, we do the inverse operation:

$$\mathbf{X}_{\text{dequant}} = \frac{\max|\mathbf{X}| \cdot \mathbf{X}_{\text{quant}}}{127}$$

This means that if we dequantize our weight, we obtain $\frac{3.2 \cdot 4}{127} \approx 0.1008$. We can see a rounding error of 0.0008 in this example. In Python, we can implement it as follows with the PyTorch library:

```
import torch
def absmax_quantize(X):
    # Calculate scale
    scale = 127 / torch.max(torch.abs(X))
    # Quantize
    X_quant = (scale * X).round()
    return X_quant.to(torch.int8)
```

Zero-point quantization, on the other hand, considers asymmetric input distributions and maps the weights $^{0.0008}$ to the range [-128, 127] by introducing a zero-point offset:

$$\mathbf{X}_{\text{quant}} = \text{round}(\text{scale} \cdot \mathbf{X} + \text{zeropoint})$$

Where $\text{scale} = \frac{255}{\max(\mathbf{X}) - \min(\mathbf{X})}$ and $\text{zeropoint} = -\text{round}(\text{scale} \cdot \min(\mathbf{X})) - 128$.

If we take the same example with a weight of 0.1, we get a scale of $\frac{255}{3.2+3.0} \approx 41.13$ and a zero-point value of $-\text{round}\left(\frac{255}{3.2+3.0} \cdot -3.0\right) - 128 = -5$. The weight of 0.1 would be quantized to $\text{round}(41.13 \cdot 0.1 - 5) = -1$, unlike the value of 4 provided by absmax.

We can easily get the dequantization by applying the inverse operation:

$$\mathbf{X}_{\text{dequant}} = \frac{\mathbf{X}_{\text{quant}} - \text{zeropoint}}{\text{scale}}$$

In Python, zero-point quantization can be implemented as follows:

```
def zeropoint_quantize(X):
    # Calculate value range (denominator)
    x_range = torch.max(X) - torch.min(X)
    x_range = 1 if x_range == 0 else x_range
    # Calculate scale
    scale = 255 / x_range
    # Shift by zero-point
    zeropoint = (-scale * torch.min(X) - 128).round()
    # Scale and round the inputs
    X_quant = torch.clip((X * scale + zeropoint).round(), -128, 127)
```

```
    return X_quant.to(torch.int8)
```

However, naïve quantization methods have limitations, particularly when dealing with *outlier features* in LLMs. Outlier features are extreme weight values (about 0.1% of total values) that can significantly impact the quantization process, leading to reduced precision for other values.

Discarding these outliers is not feasible, as it would degrade a model's performance. You can see an example of outliers in *Figure 8.11*:

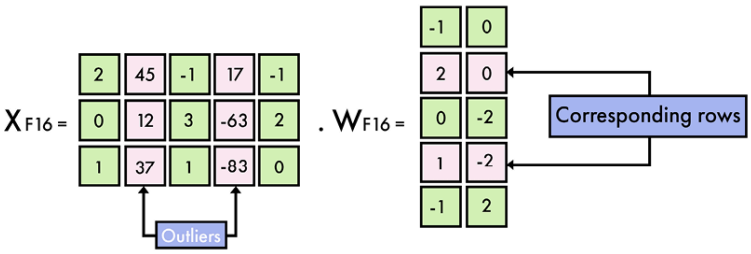

Figure 8.11 – Example of outliers in a weight matrix

To address the outlier problem, more advanced quantization techniques have been proposed. One notable example is `LLM.int8()`, introduced by Dettmers et al. (2022). `LLM.int8()` employs a mixed-precision quantization scheme, where outlier features are processed using FP16, while the remaining values are quantized to INT8. This approach effectively reduces the memory footprint of LLMs by nearly 2x while minimizing performance degradation.

`LLM.int8()` works by performing matrix multiplication in three steps. First, it extracts columns containing outlier features from the input hidden states using a custom threshold. Second, it performs separate matrix multiplications for the outliers (in `FP16`) and non-outliers (in `INT8`) using vector-wise quantization. Finally, it dequantizes the non-outlier results and combines them with the outlier results to obtain the final output in *FP16*.

The effectiveness of `LLM.int8()` has been demonstrated empirically, showing negligible performance degradation (<1%) compared to the original `FP32` models. However, it does introduce an additional computational overhead, resulting in around 20% slower inference for large models. Models can be directly loaded in 8-bit precision with the transformer library, using `LLM.int8()`, as follows:

```
from transformers import AutoModelForCausalLM
model_name = "meta-llama/Meta-Llama-3-8B-Instruct"
model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto", load_i
```

Introduced by *Dettmers et al.* (2023), NF4 is a 4-bit precision format designed for QLoRA (discussed in *Chapter 5*). It is also integrated into the transformers library but requires the bitsandbytes library as a dependency. To load a model in NF4 (4-bit precision), you can use the `load_in_4bit` parameter, as follows:

```
from transformers import AutoModelForCausalLM
model_name = "meta-llama/Meta-Llama-3-8B-Instruct"
model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto", load_i
```

Quantization with GGUF and llama.cpp

The llama.cpp project is an open-source C++ software library created by Georgi Gerganov, designed to perform inference with various LLMs. It is the most popular quantization technique, with many quantized models available on the Hugging Face Hub.

Compared to other libraries that rely on hardware-specific closed-source libraries like CUDA, llama.cpp can run on a broader range of hardware. It has gained significant popularity, particularly among users without specialized hardware, as it can operate on CPUs and Android devices. Moreover, llama.cpp can also offload layers to the GPU, accelerating inference speed. It is compatible with different inference optimization techniques, such as FlashAttention-2 and speculative decoding.

This project features its own quantization format, GGUF, designed to simplify and speed up model loading. GGUF files store tensors and metadata, supporting various formats, from 1-bit to 8-bit precision. It follows a naming convention based on the number of bits used and specific variants, such as:

- `IQ1_S` and `IQ1_M`: 1-bit precision – very low quality
- `IQ2_XXS/XS/S/M` and `Q2_K`: 2-bit precision – generally low quality but IQ2 can be usable for large models
- `IQ3_XXS/XS/S/M` and `Q3_K_S/M/L`: 3-bit precision – low quality but usable for large models
- `IQ4_XS/NL` and `Q4_K_S/M, Q4_0/1`: 4-bit precision – good quality and usable for most models
- `Q5_K_S/M` and `Q5_0/1`: 5-bit precision – high quality
- `Q6_K`: 6-bit precision –very high quality
- `Q8_0`: 8-bit precision – highest quality

To provide a brief overview of GGUF quantization, llama.cpp groups values into blocks and rounds them to a lower precision. For instance, the legacy Q4_0 format handles 32 values per block, scaling and quantizing them based on the largest weight value in the block ($w = q \times \text{block\_scale}$). In Q4_1, the smallest Lvalue in the block is also added ($w = q \times \text{block\_scale} + \text{block\_minimum}$). In Q4_K, weights are divided into super-blocks, containing 8 blocks with 32 values. Block scales and minimum values are also quantized in higher precision with 6 bits ($w = q \times \text{block\_scale(6bit)} + \text{block\_min(6bit)}$). Finally, i-quants like IQ4_XS are inspired by another quantization technique called QuIP#. This ensures an even number of positive (or negative) quant signs in groups of eight and implements the E_8 lattice to store their magnitude.

Here is a practical example of how to quantize a model in the GGUF format. The following steps can be executed on a free T4 GPU in Google Colab:

1. Install llama.cpp and the required libraries:

```
!git clone https://github.com/ggerganov/llama.cpp
!cd llama.cpp && git pull && make clean && LLAMA_CUBLAS=1 make
!pip install -r llama.cpp/requirements.txt
```

2. Download the model to convert. We will provide the model ID from the Hugging Face Hub – for example, `mistralai/Mistral-7B-Instruct-v0.2`:

```
MODEL_ID = "mlabonne/EvolCodeLlama-7b"
MODEL_NAME = MODEL_ID.split('/')[-1]
!git lfs install
!git clone https://huggingface.co/{MODEL_ID}
```

3. First, we convert the model into FP16. This is an intermediary artifact that will be used for every GGUF quantization type. Note that different conversion scripts exist in llama.cpp and are compatible with different models:

```
fp16 = f"{MODEL_NAME}/{MODEL_NAME.lower()}.fp16.bin"
!python llama.cpp/convert.py {MODEL_NAME} --outtype f16 --outfile {fp16}
```

4. We select a format (here, `Q4_K_M`) and start the quantization. This process can take an hour on a T4 GPU:

```
METHOD = "q4_k_m"
qtype = f"{MODEL_NAME}/{MODEL_NAME.lower()}.{method.upper()}.gguf"
!./llama.cpp/quantize {fp16} {qtype} {METHOD}
```

5. Once it's done, your quantized model is ready. You can download it locally, or upload it to the Hugging Face Hub using the following code:

```
from huggingface_hub import create_repo, HfApi
hf_token = "" # Specify your token
username = "" # Specify your username
api = HfApi()
# Create empty repo
create_repo(
    repo_id = f"{username}/{MODEL_NAME}-GGUF",
    repo_type="model",
    exist_ok=True,
    token=hf_token
)
# Upload gguf files
api.upload_folder(
    folder_path=MODEL_NAME,
    repo_id=f"{username}/{MODEL_NAME}-GGUF",
    allow_patterns=f"*.gguf",
    token=hf_token
)
```

GGUF models can be used with backends such as llama-cpp-python and frameworks like LangChain. This is useful if you want to integrate a quantized model into a broader system. You can also directly chat with the model using frontends, like llama.cpp's lightweight server, LM Studio, and the Text Generation Web UI. These tools enable easy interaction with the GGUF models, providing an experience similar to ChatGPT.

Quantization with GPTQ and EXL2

While GGUF and llama.cpp offer CPU inference with GPU offloading, GPTQ and EXL2 are two quantization formats dedicated to GPUs. This makes them both faster than llama.cpp during inference. In particular, EXL2 offers the highest throughput with its dedicated library, ExLlamaV2.

GPTQ and EXL2 quants are based on the GPTQ algorithm, introduced by Frantar et al. (2023). It optimizes weight quantization for LLMs by refining **the Optimal Brain Quantization** (**OBQ**) approach to handle extensive matrices efficiently. It begins with a Cholesky decomposition of the Hessian inverse, ensuring numerical stability. Instead of quantizing weights in a strict order, GPTQ processes them in batches, updating columns and associated blocks iteratively. This method leverages lazy batch updates, reducing computational redundancy and memory bottlenecks.

While GPTQ is limited to 4-bit precision, EXL2 offers more flexibility with a highly customizable precision that can mix different quantization levels. This allows for precise bitrates between 2 and 8 bits per weight, such as `2.3`, `3.5`, or `6.0`. It can also apply multiple quantization levels to each linear layer, prioritizing more important weights with higher bit quantization. Parameters are selected automatically, by quantizing each matrix multiple times and choosing a combination that minimizes the quantization error while meeting a target bitrate. In practice, this allows 70B models to run on a single 24 GB GPU with 2.55-bit precision.

The inference itself is handled by the ExLlamaV2 library, which supports both the GPTQ and EXL2 models.

In the following example, let's quantize a model in the EXL2 format using ExLlamaV2. These steps can be executed on a free T4 GPU in Google Colab:

1. Install the ExLlamaV2 library from source:

```
!git clone https://github.com/turboderp/exllamav2
!pip install -e exllamav2
```

2. We download the model to quantize by cloning its repo from the Hugging Face Hub:

```
MODEL_ID = "meta-llama/Llama-2-7b-chat-hf"
MODEL_NAME = MODEL_ID.split('/')[-1]
!git lfs install
!git clone https://huggingface.co/{MODEL_ID}
```

3. Download the calibration dataset used to measure the quantization error. In this case, we will use WikiText-103, a standard calibration dataset with high-quality articles from Wikipedia:

```
!wget https://huggingface.co/datasets/wikitext/resolve/9a9e482b5987f9d25b3a9b288
```

4. Quantize the model at a given precision (for example, 4.5):

```
!mkdir quant
!python exllamav2/convert.py \
    -i {MODEL_NAME} \
    -o quant \
    -c wikitext-test.parquet \
    -b 4.5
```

The quantized model can then be uploaded to the Hugging Face Hub, as seen previously.

GPTQ and EXL2 quants are not as widely supported as GGUF. For example, frontends like LM Studio do not currently integrate them. You can use other tools instead, like oobabooga's Text Generation Web UI. It is also directly integrated into the transformers library and supported by TGI. GPTQ models are also supported in TensorRT-LLM.

While less popular than GGUF, you can find a lot of GPTQ and EXL2 models on the Hugging Face Hub.

Other quantization techniques

There is a variety of quantization techniques beyond GGUF, GPTQ, and EXL2. This subsection will briefly introduce **Activate-aware Weight Quantization** (**AWQ**) as well as extreme quantization techniques, like QuIP# (Quantization with Incoherence Processing) and HQQ (Half-Quadratic Quantization).

Introduced by Lin et al. (2023), AWQ is another popular quantization algorithm. It identifies and protects the most important weights, which are determined based on activation magnitude instead of weight magnitude. This approach involves applying optimal per-channel scaling to these salient weights, without relying on backpropagation or reconstruction, ensuring that the LLM does not overfit the calibration set. While it relies on a different paradigm, AWQ is quite close to the GPTQ and EXL2 versions, although slightly slower. They are well-supported by inference engines and integrated into TGI, vLLM, and TensorRT-LLM.

An interesting trend is the quantization of models into 1- or 2-bit precision. While some formats, like EXL2, allow extreme quantization, the quality of the models often suffers significantly. However, recent algorithms like QuIP# and HQQ have targeted this regime and offer quantization methods that better preserve the performance of the original models. This is particularly true for large models (over 30B parameters), which can end up taking less space than 7B or 13B parameter models while providing higher-quality outputs.

This trend is expected to continue, further optimizing these quantization methods.

To conclude this chapter, here is a table summarizing the features of the three main inference engines we covered in the previous sections:

Technique	TGI	vLLM	TensorRT-LLM
Continuous batching	✓	✓	✓

Speculative decoding	✓		
FlashAttention2	✓	✓	✓
PagedAttention	✓	✓	✓
Pipeline parallelism			✓
Tensor parallelism	✓	✓	✓
GPTQ	✓		✓
EXL2	✓		
AWQ	✓	✓	✓

Table 8.1 – Summary of features for TGI, vLLM, and TensorRT-LLM

Summary

In summary, inference optimization is a critical aspect of deploying LLMs effectively. This chapter explored various optimization techniques, including optimized generation methods, model parallelism, and weight quantization. Significant speedups can be achieved by leveraging techniques like predicting multiple tokens in parallel with speculative decoding, or using an optimized attention mechanism with FlashAttention-2. Additionally, we discussed how model parallelism methods, including data, pipeline, and tensor parallelism, distribute the computational load across multiple GPUs to increase throughput and reduce latency. Weight quantization, with formats like GGUF and EXL2, further reduces the memory footprint and accelerates inference, with some calculated trade-off in output quality.

Understanding and applying these optimization strategies are essential for achieving high performance in practical applications of LLMs, such as chatbots and code completion. The choice of techniques and tools depends on specific requirements, including available hardware, desired latency, and throughput. By combining various approaches, such as continuous batching and speculative decoding, along with advanced attention mechanisms and model parallelism, users can tailor their deployment strategies to maximize efficiency.

Way back in *Chapter 4*, we focused only on implementing the ingestion pipeline, which is just one component of a standard RAG application. In the next chapter, we will conclude the RAG system by implementing the retrieval and generation components and integrating them into the inference pipeline.

References

- Hugging Face, Text Generation Inference, https://github.com/huggingface/text-generation-inference, 2022.
- *W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C.H. Yu, J.E. Gonzalez, H. Zhang, I. Stoica, Efficient Memory Management for Large Language*

Model Serving with PagedAttention, 2023.

- Nvidia, *TensorRT-LLM*, https://github.com/NVIDIA/TensorRT-LLM, 2023.
- *Y. Leviathan, M. Kalman, Y. Matias, Fast Inference from Transformers via Speculative Decoding, 2023.*
- *T. Cai, Y. Li, Z. Geng, H. Peng, J.D. Lee, D. Chen, T. Dao, Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads, 2024.*
- *W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C.H. Yu, J.E. Gonzalez, H. Zhang, I. Stoica, Efficient Memory Management for Large Language Model Serving with PagedAttention, 2023.*
- *R.Y. Aminabadi, S. Rajbhandari, M. Zhang, A.A. Awan, C. Li, D. Li, E. Zheng, J. Rasley, S. Smith, O. Ruwase, Y. He, DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale, 2022.*
- *Y. Huang, Y. Cheng, A. Bapna, O. Firat, M.X. Chen, D. Chen, H. Lee, J. Ngiam, Q.V. Le, Y. Wu, Z. Chen, GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism, 2019.*
- *K. James Reed, PiPPy: Pipeline Parallelism for PyTorch,* https://github.com/pytorch/PiPPy*, 2022.*
- *M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, B. Catanzaro, Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism, 2020.*
- *Verma and Vaidya, Mastering LLM Techniques: Inference Optimization, NVIDIA Developer Technical Blog,* https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/*, 2023.*
- *T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale, 2022.*
- *G. Gerganov, llama.cpp,* https://github.com/ggerganov/llama.cpp*, 2023.*
- *E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh, GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers, 2023.*
- *Tuboderp, exllamav2,* https://github.com/turboderp/exllamav2*, 2023.*
- *J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han, AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration, 2024.*

Join our book's Discord space

Join our community's Discord space for discussions with the authors and other readers:

https://packt.link/llmeng

Answers collapsed